МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования

«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС»»

Институт компьютерных наук

Кафедра инженерной кибернетики

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине «Современные интеллектуальные сетевые сервисы»

Проектирование AI-сервиса на FastAPI

Выполнил: студент группы МПИ-25-1-1
Цыканов Артём Эдуардович

Проверила: Корнет Мария Евгеньевна

Москва, 2026

Содержание

1 Цель работы **2**

2 Архитектура сервиса **2**

3 Реализованные endpoints **2**

4 Документация и проверка через Swagger UI **3**

4.1 Успешное выполнение прогноза 3

4.2 Ошибка структурной валидации 4

4.3 Получение отсутствующего прогноза 5

5 Тестирование API **6**

6 Анализ API-контракта **7**

7 Вывод **7**

1 Цель работы

Изучить принципы проектирования REST API для AI-сервиса, реализовать контур инференса на FastAPI, описать контракт при помощи Pydantic и OpenAPI, а также проверить корректную обработку запросов и ошибок.

2 Архитектура сервиса

Сервис работает синхронно: клиент отправляет данные в формате JSON и получает оценку риска в рамках одного HTTP-запроса. Pydantic проверяет структуру данных, сервисный слой контролирует бизнес-ограничения, условная ML-модель рассчитывает числовую оценку риска, а постобработка формирует категорию и рекомендацию. Полученный результат сохраняется в оперативной памяти.

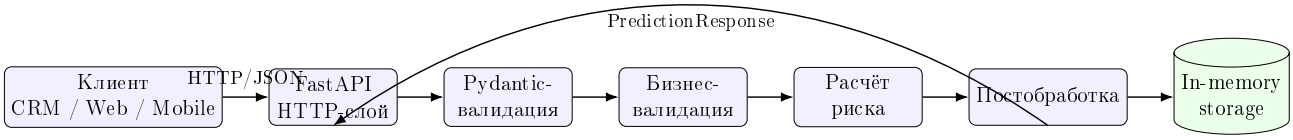


Рис. 1: Компонентная схема сервиса агроскоринга

Приложение разделено на следующие программные компоненты:

- `main.py` — HTTP-слой, middleware и маршруты FastAPI;
- `schemas.py` — Pydantic-схемы запросов и ответов;
- `model.py` — функция расчёта риска;
- `services.py` — бизнес-валидация и постобработка;
- `storage.py` — временное in-memory хранилище прогнозов.

Исходный код решения размещён в репозитории на GitHub.

3 Реализованные endpoints

Таблица 1: Endpoints сервиса

Endpoint	Метод	Назначение
`/health`	GET	Проверка работоспособности HTTP API
`/model-info`	GET	Получение метаданных и статуса готовности модели
`/predict`	POST	Валидация, инференс и сохранение прогноза
`/predictions/{request_id}`	GET	Поиск сохранённого прогноза по UUID
`/predictions`	GET	Список прогнозов с фильтром и ограничением количества

Endpoint `POST /predict` возвращает `200 OK`, поскольку его основная семантика — синхронное вычисление результата. Внутреннее сохранение прогноза не делает этот endpoint классической операцией создания ресурса со статусом `201 Created`.

Endpoint `/health` проверяет доступность самого HTTP API, а `/model-info` сообщает сведения о конкретной модели и её готовности. Поэтому API может оставаться доступным, когда модель временно не готова к инференсу.

4 Документация и проверка через Swagger UI

FastAPI автоматически сформировал OpenAPI-описание сервиса и интерактивную документацию Swagger UI. В документации представлены все пять маршрутов, Pydantic-схемы, ограничения параметров и возможные HTTP-коды.

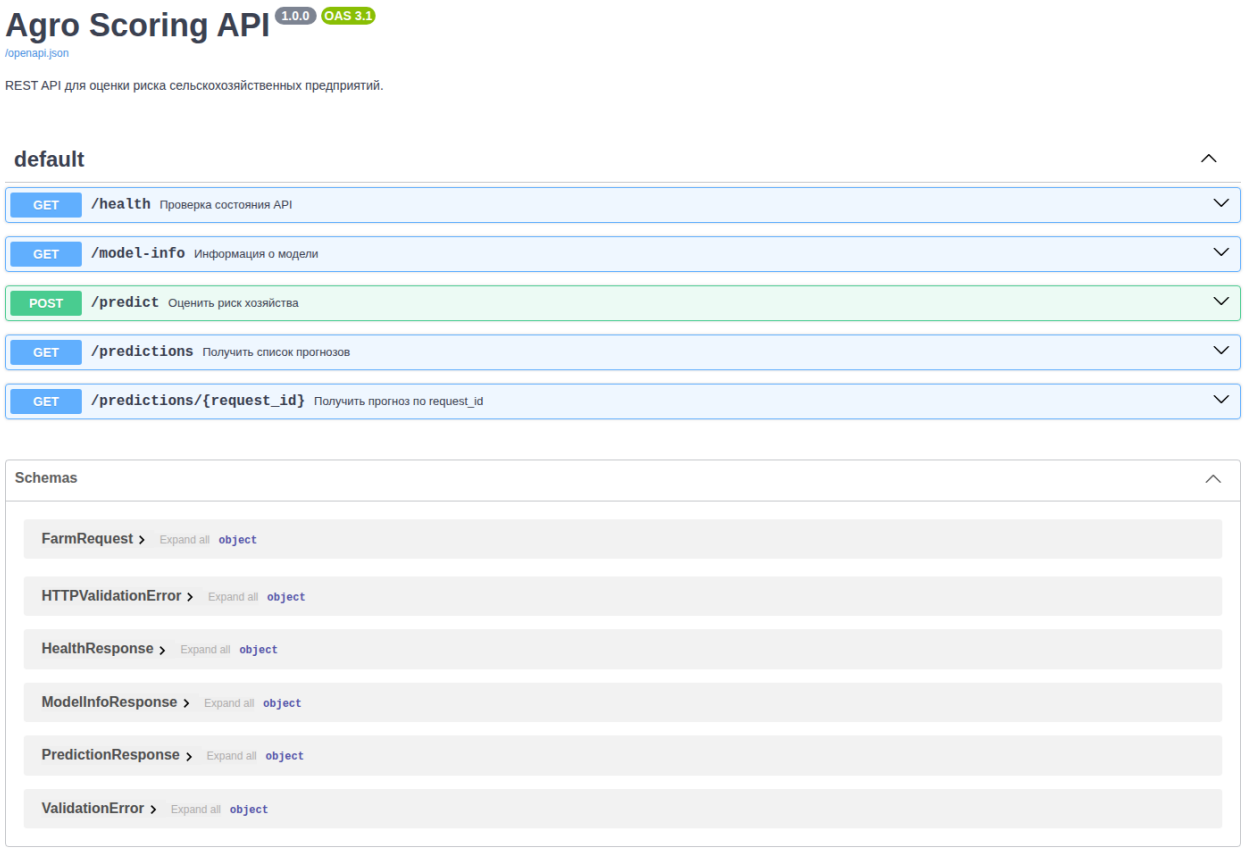


Рис. 2: Общий вид Swagger UI

4.1 Успешное выполнение прогноза

Для корректного запроса сервис рассчитал оценку риска `0.9`, присвоил категорию `high`, сформировал рекомендацию и вернул ответ со статусом 200.

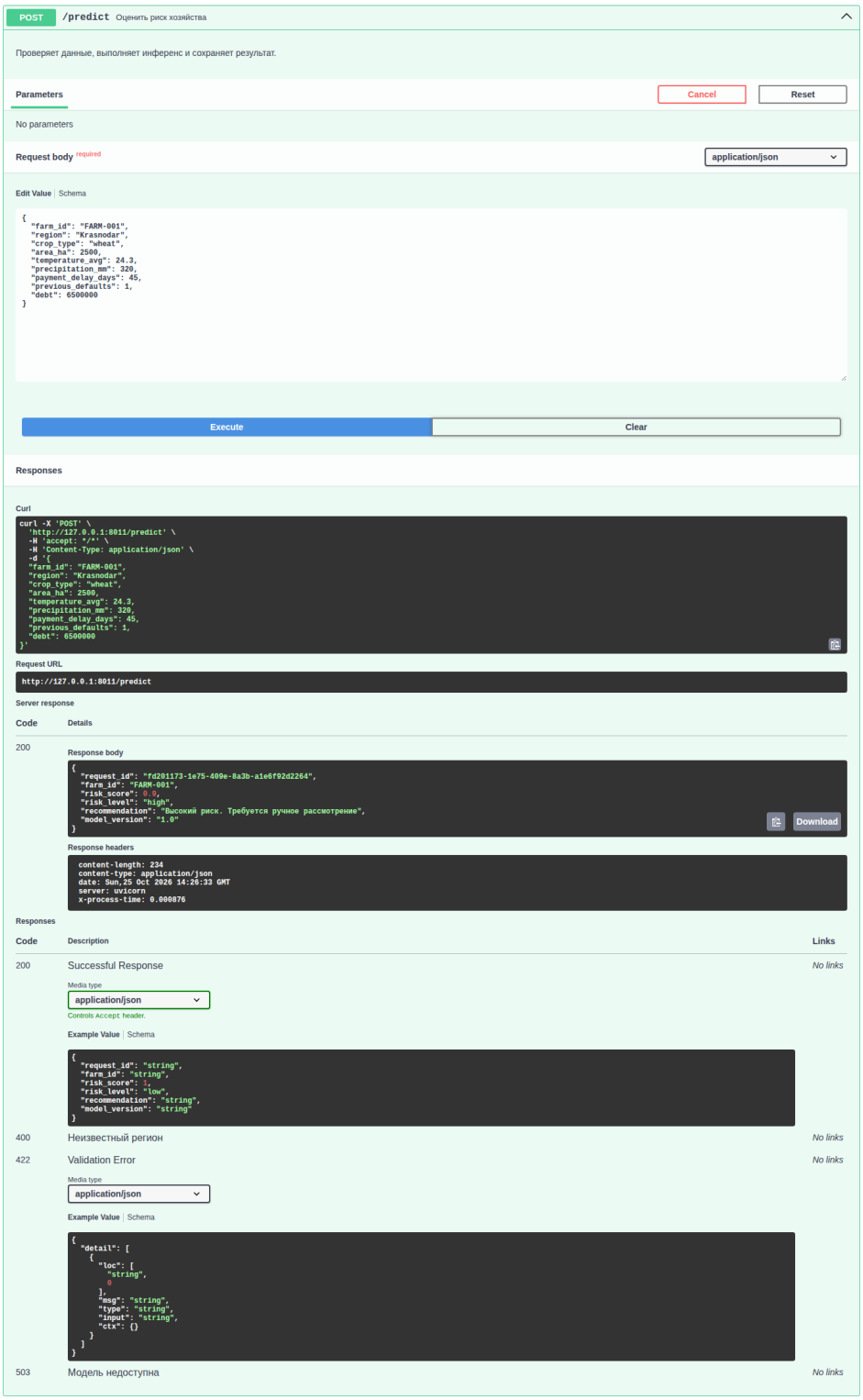


Рис. 3: Успешный вызов POST /predict

4.2 Ошибка структурной валидации

Запрос с отрицательной площадью не должен доходить до ML-модели: такое значение нарушает контракт, делает прогноз бессмысленным и напрасно расходует вычислительные ресурсы. FastAPI отклоняет запрос на границе сервиса со статусом `422 Unprocessable Entity`.

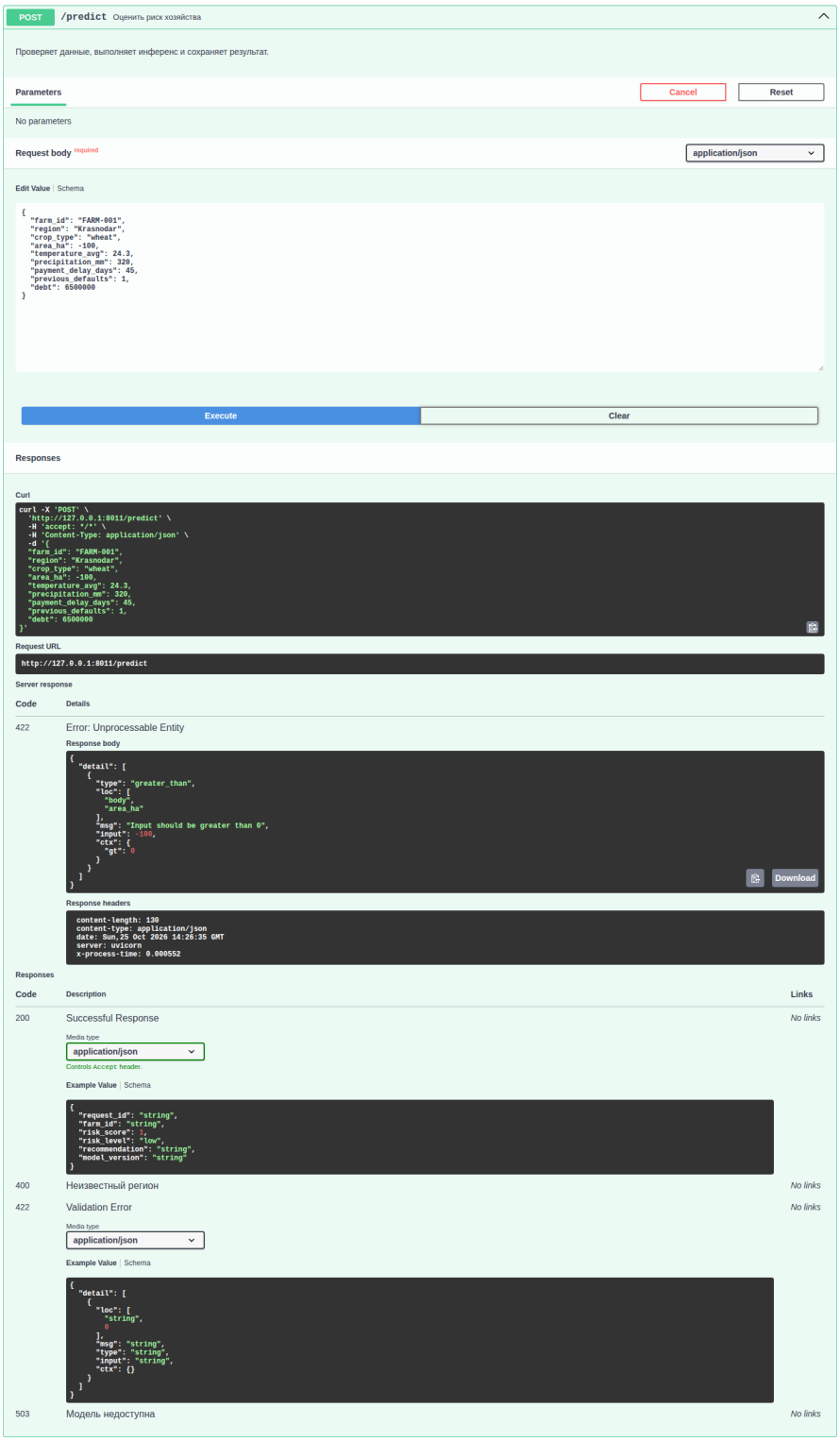


Рис. 4: Ошибка Pydantic при отрицательной площади посевов

4.3 Получение отсутствующего прогноза

При обращении к неизвестному `request_id` сервис возвращает статус `404 Not Found` и диагностическое сообщение `Prediction not found`.

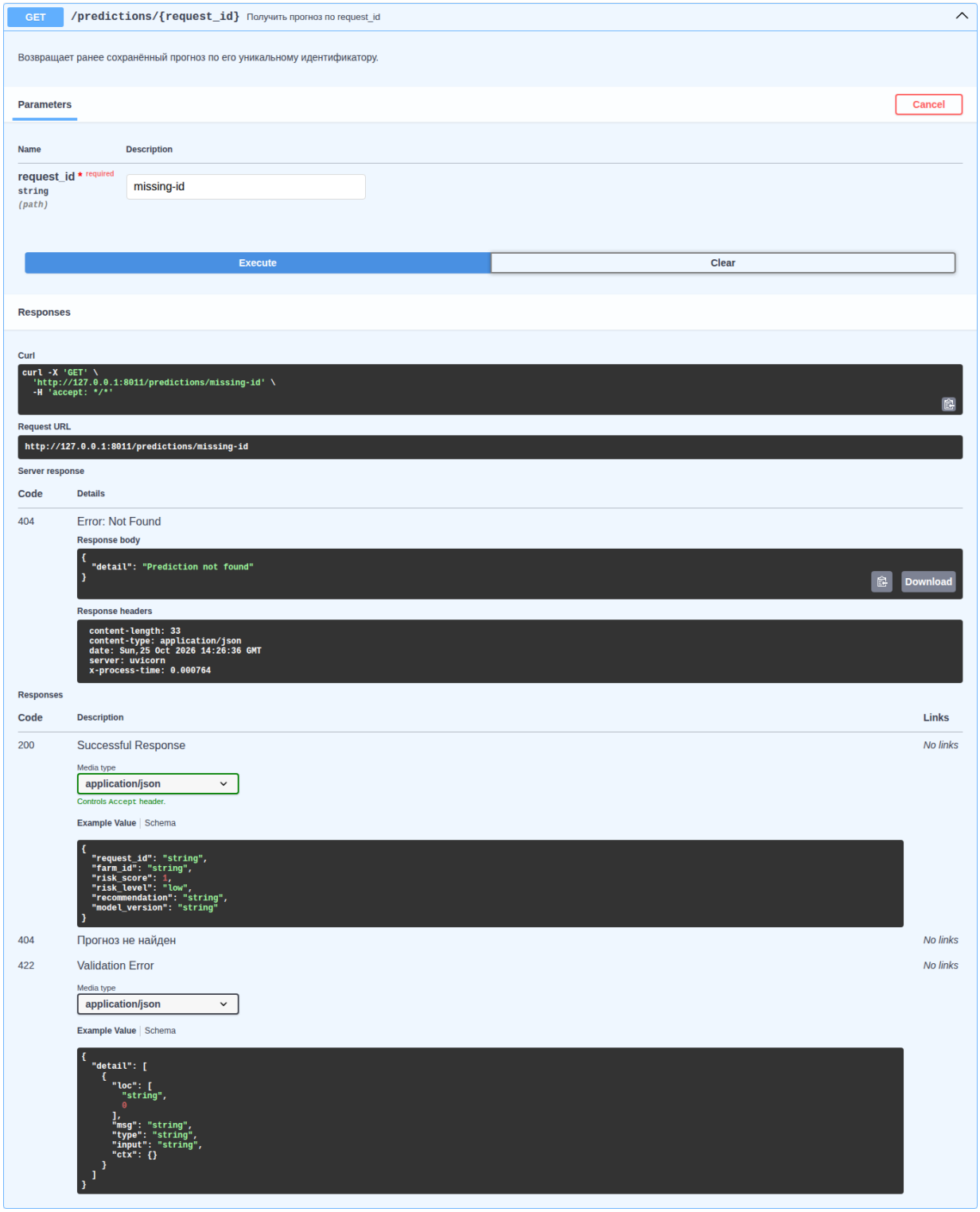


Рис. 5: Ответ 404 для неизвестного request_id

5 Тестирование API

Таблица 2: Результаты обязательных тест-кейсов

№	Запрос	Ожидаемый код	Полученный код	Результат
1	`GET /health`	200	200	Пройден
2	`GET /model-info`	200	200	Пройден
3	Корректный `POST /predict`	200	200	Пройден
4	`area_ha = -100`	422	422	Пройден
5	Неизвестный регион	400	400	Пройден
6	Существующий `request_id`	200	200	Пройден
7	Неизвестный `request_id`	404	404	Пройден
8	`limit = 2`	200	200	Пройден
9	`risk_level = high`	200	200	Пройден
10	`limit = -5`	422	422	Пройден

Автоматические тесты дополнительно проверяют границы категорий риска, недопустимое значение `risk_level`, ответ 503 при недоступной модели, формат UUID, наличие заголовка `X-Process-Time`, OpenAPI-схемы и доступность Swagger UI. Все 20 тестов пройдены успешно.

6 Анализ API-контракта

Таблица 3: Анализ API-контракта

Endpoint	Метод	Вход	Выход	Успех	Ошибки
`/health`	GET	—	HealthResponse	200	500
`/model-info`	GET	—	ModelInfoResponse	200	500
`/predict`	POST	JSON FarmRequest	PredictionResponse	200	400, 422, 503, 500
`/predictions/{request_id}`	GET	path: request_id	PredictionResponse	200	404, 500
`/predictions`	GET	query: limit, risk_level	PredictionResponse[]	200	422, 500

OpenAPI-документ явно описывает пути, HTTP-методы, обязательные поля, ограничения и схемы ответов. Благодаря параметру `response_model` внешний контракт не зависит от внутреннего формата хранения данных.

7 Вывод

В ходе работы разработан модульный FastAPI-сервис агроскоринга. Сервис валидирует входные данные, рассчитывает и сохраняет прогнозы, поддерживает поиск и фильтрацию, возвращает семантически корректные HTTP-коды и публикует OpenAPI-контракт.

REST API в AI-системе является стабильной границей между моделью и клиентами: он скрывает детали реализации модели, задаёт форматы обмена и правила обработки

ошибок, а также позволяет менять внутреннюю реализацию без изменения внешних информационных систем.